

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО»
(Университет ИТМО)

Факультет **Прикладной информатики**
Образовательная программа **Мобильные и сетевые технологии**
Направление подготовки(специальность) **09.03.03 Прикладная информатика**

ЛАБОРАТОРНАЯ РАБОТА №5

По дисциплине «Проектирование и реализация баз данных»

Тема: Процедуры, функции, триггеры в PostgreSQL

Выполнил	Мезенцев Б.Г.; К 3239.
Проверил	Говорова М.М.
Дата	13.05.2026

Санкт-Петербург 2026

1 Цель работы

Овладеть практическими навыками создания и использования процедур, функций и триггеров в базе данных PostgreSQL.

2 Практическое задание

- Создать три процедуры для индивидуальной базы данных согласно варианту.
- Создать оригинальные триггеры для индивидуальной базы данных.
- Выполнить тестирование разработанных объектов в среде SQL Shell (psql).
- Подготовить подтверждающие скриншоты кода и результатов работы процедур и триггеров.

3 Схема базы данных

Вариант: 17 — БД «Телефонный провайдер».

Наименование базы данных: Lab_db.

Схема базы данных: phone_provider.

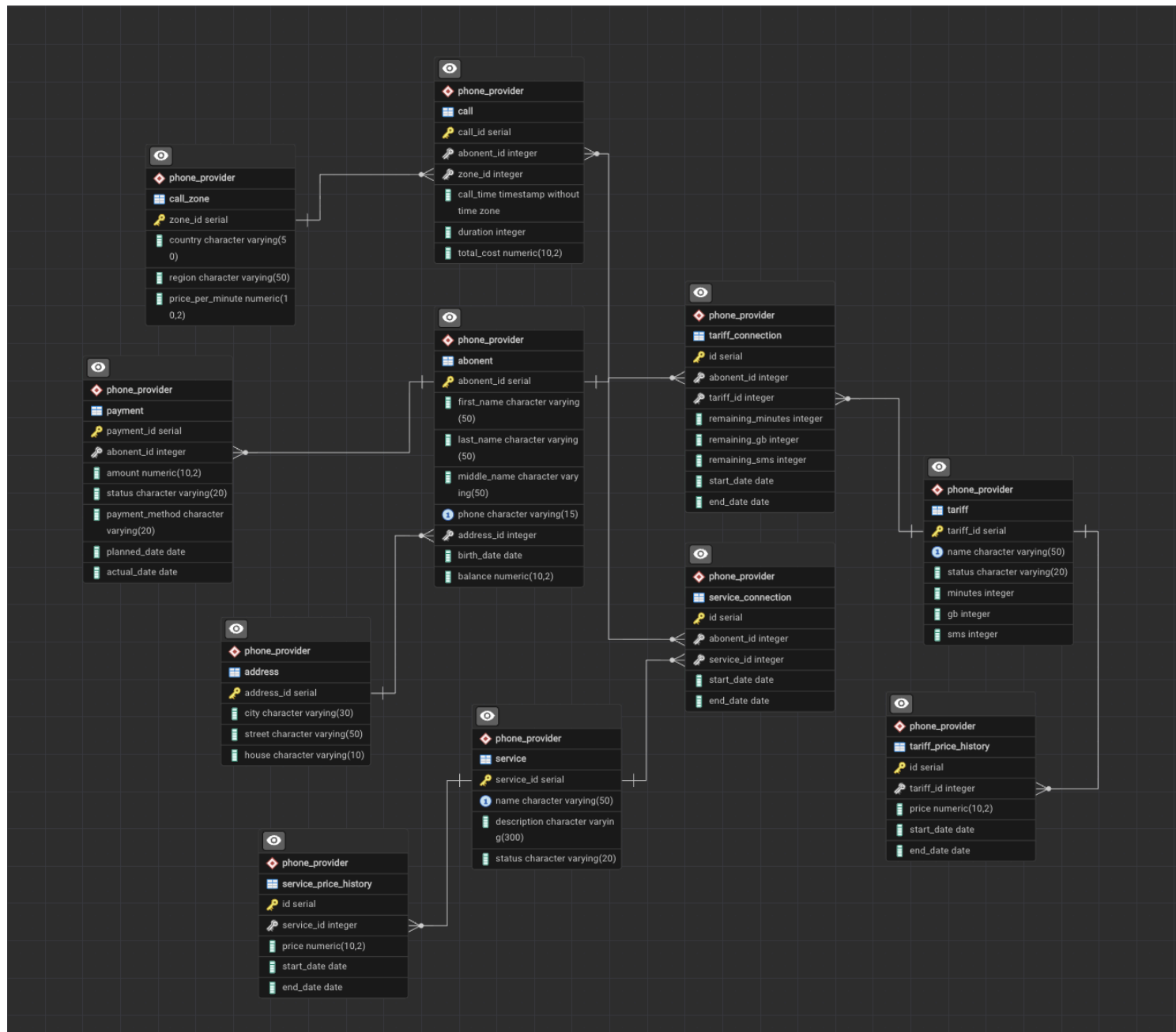


Рис. 1: ERD-диаграмма базы данных из ЛР3

4 Выполнение лабораторной работы №5

4.1 Разработка процедур

Процедура 1. Добавить данные о новом звонке абонента.

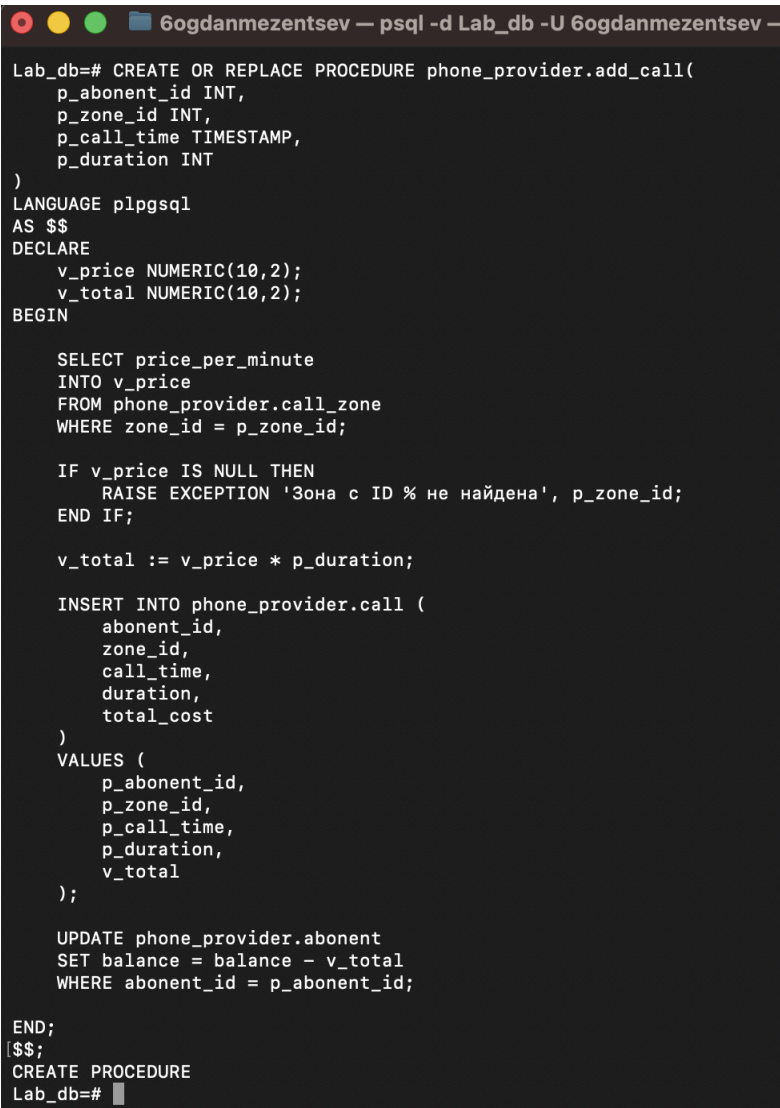
Процедура `add_call` добавляет новый звонок абонента в таблицу `call`.

Что делает:

- получает цену минуты для выбранной зоны
- рассчитывает стоимость звонка
- добавляет запись в таблицу `call`
- списывает стоимость звонка с баланса абонента

Входные параметры:

- ID абонента
- ID зоны
- дата и время звонка
- продолжительность разговора



```
6ogdanmezentsev — psql -d Lab_db -U 6ogdanmezentsev —
Lab_db=# CREATE OR REPLACE PROCEDURE phone_provider.add_call(
    p_abonent_id INT,
    p_zone_id INT,
    p_call_time TIMESTAMP,
    p_duration INT
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_price NUMERIC(10,2);
    v_total NUMERIC(10,2);
BEGIN

    SELECT price_per_minute
    INTO v_price
    FROM phone_provider.call_zone
    WHERE zone_id = p_zone_id;

    IF v_price IS NULL THEN
        RAISE EXCEPTION 'Зона с ID % не найдена', p_zone_id;
    END IF;

    v_total := v_price * p_duration;

    INSERT INTO phone_provider.call (
        abonent_id,
        zone_id,
        call_time,
        duration,
        total_cost
    )
    VALUES (
        p_abonent_id,
        p_zone_id,
        p_call_time,
        p_duration,
        v_total
    );

    UPDATE phone_provider.abonent
    SET balance = balance - v_total
    WHERE abonent_id = p_abonent_id;

END;
[$$;
CREATE PROCEDURE
Lab_db=#
```

Рис. 2: Код процедуры 1

```
Lab_db=# SELECT *
FROM phone_provider.abonent
WHERE abonent_id = 1;
[ abonent_id | first_name | last_name | middle_name | phone | address_id | birth_date | balance
-----+-----+-----+-----+-----+-----+-----+-----
          1 | Иван      | Иванов   | Иванович   | 9000000001 |          1 | 1995-05-10 | 460.00
(1 row)

Lab_db=# SELECT *
FROM phone_provider.call
ORDER BY call_id DESC
[LIMIT 5;
 call_id | abonent_id | zone_id | call_time | duration | total_cost
-----+-----+-----+-----+-----+-----
          3 |          2 |          3 | 2026-04-03 18:30:00 |          3 |          30.00
          2 |          1 |          2 | 2026-04-02 12:00:00 |          5 |          15.00
          1 |          1 |          1 | 2026-04-01 10:00:00 |         10 |          25.00
(3 rows)
```

Рис. 3: Данные в таблицах abonent и call до вызова процедуры

```
Lab_db=# CALL phone_provider.add_call(
          1,
          2,
          NOW()::timestamp,
          15
);
CALL
```

Рис. 4: Вызов процедуры add_call

```
Lab_db=# SELECT *
FROM phone_provider.abonent
[WHERE abonent_id = 1;
 abonent_id | first_name | last_name | middle_name | phone | address_id | birth_date | balance
-----+-----+-----+-----+-----+-----+-----+-----
          1 | Иван      | Иванов   | Иванович   | 9000000001 |          1 | 1995-05-10 | 415.00
(1 row)

Lab_db=# SELECT *
FROM phone_provider.call
ORDER BY call_id DESC
[LIMIT 5;
 call_id | abonent_id | zone_id | call_time | duration | total_cost
-----+-----+-----+-----+-----+-----
          4 |          1 |          2 | 2026-05-13 14:18:43.627817 |         15 |          45.00
          3 |          2 |          3 | 2026-04-03 18:30:00 |          3 |          30.00
          2 |          1 |          2 | 2026-04-02 12:00:00 |          5 |          15.00
          1 |          1 |          1 | 2026-04-01 10:00:00 |         10 |          25.00
(4 rows)
```

Рис. 5: Данные в таблицах abonent и call после вызова процедуры

Процедура 2. Вывести задолженность по оплате для заданного абонента.

Процедура `get_debt` вычисляет задолженность абонента.

Что делает:

- получает текущий баланс абонента
- если баланс отрицательный — возвращает размер долга
- если баланс положительный — возвращает 0

Входной параметр:

- ID абонента

```

Lab_db=# CREATE OR REPLACE PROCEDURE get_debt(
    IN p_abonent_id INT,
    OUT p_debt NUMERIC(10,2)
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_balance NUMERIC(10,2);
BEGIN

    SELECT balance
    INTO v_balance
    FROM phone_provider.abonent
    WHERE abonent_id = p_abonent_id;

    IF v_balance < 0 THEN
        p_debt := ABS(v_balance);
    ELSE
        p_debt := 0;
    END IF;

END;
[ $$;
CREATE PROCEDURE
Lab_db=# █

```

Рис. 6: Код процедуры 2

```

Lab_db=# UPDATE phone_provider.abonent
    SET balance = -50
    WHERE abonent_id = 1;
UPDATE 1
Lab_db=# SELECT *
FROM phone_provider.abonent
WHERE abonent_id = 1;

```

abonent_id	first_name	last_name	middle_name	phone	address_id	birth_date	balance
1	Иван	Иванов	Иванович	9000000001	1	1995-05-10	-50.00

(1 row)

Рис. 7: Данные абонента с id = 1

```

[Lab_db=# CALL get_debt(1, NULL);
p_debt
-----
50.00
(1 row)

```

Рис. 8: Вызов процедуры get_debt

Процедура 3. Рассчитать общую стоимость звонков по каждой зоне за истекшую неделю.

Процедура `weekly_zone_stats` формирует статистику стоимости звонков по зонам за последние 7 дней.

Что делает:

- выбирает звонки за последнюю неделю
- группирует их по зонам
- вычисляет суммарную стоимость звонков для каждой зоны

Результат:

- ID зоны
- страна

- общая стоимость звонков

```

Lab_db=# CREATE OR REPLACE PROCEDURE phone_provider.weekly_zone_stats()
LANGUAGE plpgsql
AS $$
BEGIN

    CREATE TEMP TABLE IF NOT EXISTS weekly_stats AS
    SELECT
        cz.zone_id,
        cz.country,
        SUM(c.total_cost) AS total_calls_cost
    FROM phone_provider.call c
    JOIN phone_provider.call_zone cz
        ON c.zone_id = cz.zone_id
    WHERE c.call_time >= CURRENT_DATE - INTERVAL '7 days'
    GROUP BY cz.zone_id, cz.country
    ORDER BY total_calls_cost DESC;

END;
[$$;
CREATE PROCEDURE
Lab_db=# █

```

Рис. 9: Код процедуры 3

```

[Lab_db=# CALL phone_provider.weekly_zone_stats();
CALL
[Lab_db=# SELECT * FROM weekly_stats;
  zone_id | country | total_calls_cost
-----+-----+-----
        2 | Россия |          45.00
(1 row)

Lab_db=# █

```

Рис. 10: Вызов процедуры weekly_zone_stats

4.2 Разработка триггеров

Триггер 1. Автоматический расчет стоимости звонка.

Перед вставкой записи в таблицу звонков триггер берет цену минуты из справочника зон(call_zone) и умножает на длительность, записывая итог в total_cost.

```
Lab_db=# CREATE OR REPLACE FUNCTION phone_provider.fn_calculate_call_cost()
RETURNS TRIGGER AS $$
BEGIN
    SELECT price_per_minute INTO NEW.total_cost
    FROM phone_provider.call_zone
    WHERE zone_id = NEW.zone_id;

    NEW.total_cost := NEW.total_cost * NEW.duration;
    RETURN NEW;
END;
[$$ LANGUAGE plpgsql;
CREATE FUNCTION
```

Рис. 11: Код триггерной функции для триггера 1

```
Lab_db=# CREATE TRIGGER tg_calculate_call_cost
BEFORE INSERT ON phone_provider.call
[FOR EACH ROW EXECUTE FUNCTION phone_provider.fn_calculate_call_cost();
CREATE TRIGGER
Lab_db=# █
```

Рис. 12: Создание триггера 1

```
[Lab_db=# SELECT * FROM phone_provider.call;
 call_id | abonent_id | zone_id | call_time | duration | total_cost
-----+-----+-----+-----+-----+-----
      1 |          1 |        1 | 2026-04-01 10:00:00 |        10 |      25.00
      2 |          1 |        2 | 2026-04-02 12:00:00 |         5 |      15.00
      3 |          2 |        3 | 2026-04-03 18:30:00 |         3 |      30.00
      4 |          1 |        2 | 2026-05-13 14:18:43.627817 |        15 |      45.00
(4 rows)
```

Рис. 13: Данные в таблице звонков до добавления нового звонка

```
Lab_db=# INSERT INTO phone_provider.call (abonent_id, zone_id, duration, call_time, total_cost)
VALUES (1, 1, 10, NOW(), 0);
INSERT 0 1
```

Рис. 14: Добавляем новый звонок для абонента с id = 1 и указываем цену звонка = 0


```

[Lab_db=# SELECT * FROM phone_provider.call;
 call_id | abonent_id | zone_id | call_time | duration | total_cost
-----+-----+-----+-----+-----+-----
      1 |          1 |        1 | 2026-04-01 10:00:00 |        10 |       25.00
      2 |          1 |        2 | 2026-04-02 12:00:00 |         5 |       15.00
      3 |          2 |        3 | 2026-04-03 18:30:00 |         3 |       30.00
      4 |          1 |        2 | 2026-05-13 14:18:43.627817 |        15 |       45.00
      5 |          1 |        1 | 2026-05-13 16:02:04.282892 |        20 |       50.00
      6 |          1 |        1 | 2026-05-13 16:04:11.348851 |        10 |       25.00
(6 rows)

```

Рис. 15: Новый звонок добавлен и стоимость рассчитана автоматически с помощью триггера

Триггер 2. Списание денег с баланса абонента после звонка.

После добавления звонка сумма его стоимости автоматически вычитается из текущего баланса абонента.

```

Lab_db=# CREATE OR REPLACE FUNCTION phone_provider.fn_deduct_balance()
 RETURNS TRIGGER AS $$
 BEGIN
   UPDATE phone_provider.abonent
   SET balance = balance - NEW.total_cost
   WHERE abonent_id = NEW.abonent_id;
   RETURN NEW;
 END;
[$$ LANGUAGE plpgsql;
CREATE FUNCTION
Lab_db=# CREATE TRIGGER tg_deduct_balance
 AFTER INSERT ON phone_provider.call
[FOR EACH ROW EXECUTE FUNCTION phone_provider.fn_deduct_balance();
CREATE TRIGGER
Lab_db=# █

```

Рис. 16: Создание триггерной функции и триггера 2

```

[Lab_db=# SELECT balance FROM phone_provider.abonent WHERE abonent_id = 1;
 balance
-----
    100.00
(1 row)

```

Рис. 17: Баланс абонента с id = 1 до срабатывания триггера 2

```

Lab_db=# INSERT INTO phone_provider.call (abonent_id, zone_id, duration, call_time)
[VALUES (1, 1, 20, NOW());
INSERT 0 1

```

Рис. 18: Добавляем новый звонок

```

Lab_db=# SELECT balance FROM phone_provider.abonent WHERE abonent_id = 1;
 balance
-----
    50.00
(1 row)

```

Рис. 19: Стоимость звонка была списана с баланса

Триггер 3. Пополнение баланса при оплате.

Триггер отслеживает изменение статуса в таблице `payment`. Если статус меняется на «paid», сумма добавляется к балансу абонента.

```

Lab_db=# CREATE OR REPLACE FUNCTION phone_provider.fn_top_up_balance()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.status = 'paid' AND (OLD.status IS DISTINCT FROM 'paid') THEN
        UPDATE phone_provider.abonent
        SET balance = balance + NEW.amount
        WHERE abonent_id = NEW.abonent_id;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
CREATE FUNCTION
Lab_db=# CREATE TRIGGER tg_top_up_balance
AFTER UPDATE ON phone_provider.payment
FOR EACH ROW EXECUTE FUNCTION phone_provider.fn_top_up_balance();
CREATE TRIGGER
Lab_db=# █

```

Рис. 20: Создание триггерной функции и триггера 3

```

Lab_db=# SELECT balance FROM phone_provider.abonent WHERE abonent_id = 1;
[SELECT payment_id, amount, status FROM phone_provider.payment WHERE abonent_id = 1 AND status = 'pending';
 balance
-----
    50.00
(1 row)

 payment_id | amount | status
-----+-----+-----
          4 | 500.00 | pending
(1 row)

```

Рис. 21: Проверяем баланс абонента и наличие неоплаченного платежа

```

Lab_db=# UPDATE phone_provider.payment
SET status = 'paid'
[WHERE payment_id = 4;
UPDATE 1

```

Рис. 22: Обновляем статус платежа на paid

```

[Lab_db=# SELECT payment_id, status, actual_date FROM phone_provider.payment ORDER BY payment_id DESC LIMIT 1;
 payment_id | status | actual_date
-----+-----+-----
          4 | paid   |
(1 row)

[Lab_db=# SELECT balance FROM phone_provider.abonent WHERE abonent_id = 1;
 balance
-----
    550.00
(1 row)

```

Рис. 23: Баланс увеличился на 500.00

Триггер 4. Установка даты фактической оплаты.

Если при обновлении платежа статус меняется на 'paid', триггер автоматически проставляет текущую дату в поле actual_date.

```

[Lab_db=# CREATE OR REPLACE FUNCTION phone_provider.fn_set_payment_date()
 RETURNS TRIGGER AS $$
 BEGIN
     IF NEW.status = 'paid' AND NEW.actual_date IS NULL THEN
         NEW.actual_date := CURRENT_DATE;
     END IF;
     RETURN NEW;
 END;
[$$ LANGUAGE plpgsql;
CREATE FUNCTION
[Lab_db=# CREATE TRIGGER tg_set_payment_date
 BEFORE UPDATE ON phone_provider.payment
[FOR EACH ROW EXECUTE FUNCTION phone_provider.fn_set_payment_date();
CREATE TRIGGER
[Lab_db=# █

```

Рис. 24: Создание триггерной функции и триггера 4

```

[Lab_db=# SELECT balance FROM phone_provider.abonent WHERE abonent_id = 2;
 balance
-----
    120.00
(1 row)

[Lab_db=# SELECT payment_id, amount, status FROM phone_provider.payment WHERE abonent_id = 2 AND status = 'pending';
 payment_id | amount | status
-----+-----+-----
          2 | 300.00 | pending
(1 row)

```

Рис. 25: Проверяем баланс и наличие неоплаченного счета

```

[Lab_db=# UPDATE phone_provider.payment SET status = 'paid' WHERE payment_id = 2;
UPDATE 1

```

Рис. 26: Переводим платеж в статус 'paid'

```
[Lab_db=# SELECT status, actual_date FROM phone_provider.payment WHERE payment_id = 2;
status | actual_date
-----+-----
paid   | 2026-05-13
(1 row)
```

Рис. 27: Поле `actual_date` заполнилось текущей датой

Триггер 5. Валидация длительности звонка.

Дополнительная валидация данных перед вставкой или обновлением в `call`. Хотя в схеме есть CHECK, триггер позволяет выдать более информативное сообщение об ошибке.

```
Lab_db=# CREATE OR REPLACE FUNCTION phone_provider.fn_validate_call()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.duration <= 0 THEN
        RAISE EXCEPTION 'Длительность звонка должна быть положительной. Введено: %', NEW.duration;
    END IF;
    RETURN NEW;
END;
[$$ LANGUAGE plpgsql;
CREATE FUNCTION
Lab_db=# CREATE TRIGGER tg_validate_call
BEFORE INSERT OR UPDATE ON phone_provider.call
FOR EACH ROW EXECUTE FUNCTION phone_provider.fn_validate_call();
CREATE TRIGGER
Lab_db=#
```

Рис. 28: Создание триггерной функции и триггера 5

```
[Lab_db=# INSERT INTO phone_provider.call (abonent_id, zone_id, duration, call_time)
VALUES (1, 1, -5, NOW());
ERROR:  Длительность звонка должна быть положительной. Введено: -5
CONTEXT:  _PL/pgSQL function phone_provider.fn_validate_call() line 4 at RAISE
```

Рис. 29: Пробуем вставить ошибочный звонок и получаем ошибку

Видим, что вернулась ожидаемая ошибка: *"Длительность звонка должна быть положительно. Введено: -5"*. Поэтому некорректные данные не будут записаны.

Триггер 6. Инициализация лимитов тарифа.

При подключении нового тарифа триггер копирует стандартные объемы минут/ГБ из описания тарифа в личную информацию абонента (остатки).

```
Lab_db=# CREATE OR REPLACE FUNCTION phone_provider.fn_init_limits()
RETURNS TRIGGER AS $$
BEGIN
    SELECT minutes, gb, sms
    INTO NEW.remaining_minutes, NEW.remaining_gb, NEW.remaining_sms
    FROM phone_provider.tariff
    WHERE tariff_id = NEW.tariff_id;
    RETURN NEW;
END;
[$$ LANGUAGE plpgsql;
CREATE FUNCTION
Lab_db=# CREATE TRIGGER tg_init_limits
BEFORE INSERT ON phone_provider.tariff_connection
[FOR EACH ROW EXECUTE FUNCTION phone_provider.fn_init_limits();
CREATE TRIGGER
Lab_db=#
```

Рис. 30: Создание триггерной функции и триггера 6

```
Lab_db=# SELECT tariff_id, minutes, gb FROM phone_provider.tariff WHERE tariff_id = 2;
tariff_id | minutes | gb
-----+-----+-----
2 | 1000 | 50
(1 row)
```

Рис. 31: Проверяем лимиты в справочнике тарифов

```
Lab_db=# INSERT INTO phone_provider.tariff_connection (abonent_id, tariff_id, start_date)
[VALUES (1, 2, CURRENT_DATE);
INSERT 0 1
```

Рис. 32: Подключаем этот тариф с tariff_id = 2 абоненту с id = 1

```
Lab_db=# SELECT remaining_minutes, remaining_gb FROM phone_provider.tariff_connection
[WHERE abonent_id = 1 AND tariff_id = 2;
remaining_minutes | remaining_gb
-----+-----
1000 | 50
(1 row)
```

Рис. 33: Поля remaining_minutes и remaining_gb заполнились автоматически

Триггер 7. Блокировка звонков при долге.

Для предотвращения роста задолженности, этот триггер блокирует возможность совершения звонка, если баланс абонента опустился ниже установленного лимита (например, -500).

```
Lab_db=# CREATE OR REPLACE FUNCTION phone_provider.fn_check_balance_before_call()
RETURNS TRIGGER AS $$
DECLARE
    v_balance NUMERIC;
BEGIN
    SELECT balance INTO v_balance FROM phone_provider.abonent WHERE abonent_id = NEW.abonent_id;
    IF v_balance < -500 THEN
        RAISE EXCEPTION 'Звонок невозможен: критический баланс (%)', v_balance;
    END IF;
    RETURN NEW;
END;
[Lab_db=# LANGUAGE plpgsql;
CREATE FUNCTION
Lab_db=# CREATE TRIGGER tg_check_balance_before_call
BEFORE INSERT ON phone_provider.call
FOR EACH ROW EXECUTE FUNCTION phone_provider.fn_check_balance_before_call();
CREATE TRIGGER
Lab_db=# ]
```

Рис. 34: Создание триггерной функции и триггера 7

```
[Lab_db=# SELECT balance FROM phone_provider.abonent WHERE abonent_id = 2;
 balance
-----
-600.00
(1 row)
```

Рис. 35: Проверяем, что у абонента большой долг(-600.00)

```
[Lab_db=# INSERT INTO phone_provider.call (abonent_id, zone_id, duration, call_time)
VALUES (2, 1, 10, NOW());
ERROR:  Звонок невозможен: критический баланс (-600.00)
CONTEXT:  PL/pgSQL function phone_provider.fn_check_balance_before_call() line 7 at RAISE
```

Рис. 36: Пробуем добавить звонок для этого абонента и получаем ошибку

Видим, что вернулась ожидаемая ошибка: *"Звонок невозможен: критический баланс (-600.00)"*. Поэтому запись не добавится.

5 Вывод

В ходе выполнения лабораторной работы были разработаны и протестированы три процедуры и семь триггеров для базы данных PostgreSQL для предметной области «Телефонный провайдер».

Разработанные триггеры позволили поддерживать согласованность данных между связанными таблицами, автоматически изменять статусы платежей, пересчитывать баланс абонента, а также контролировать корректность вставки новых записей. Запросы в SQL Shell (psql) показали, что все созданные объекты работают корректно.